

Just Hexagons

A bundle full of useful scripts for working with hexagons in every way.

Author: Radek Titěra

Contents

- Just Hexagons
 - Contents
 - Setup/Usage
 - Scripts
 - * Hexagon
 - * Geometry 2D
 - * Geometry 3D
 - * Navigation
 - Relations
 - Line
 - * Grid
 - * Map
 - * Editor
 - * Debug
 - * IO
 - Binary
 - * Utils
 - Placement Plane
 - Mesh Builder

Setup/Usage

No setup is needed. Scripts are ready to use.

See the Demos directory for example uses of various features or the Scripts section below for a more detailed overview.

Scripts

Hexagon

The **Hexagon** is the core of this library.

All hexagons enforce $Q + R + S = 0$

A **Hexagon** is defined as an immutable **record** of **axial coordinates** **Q**, **R**, and **S**.

Hexagon's hash is the combination of **Q**, **R**, and **S**, so two hexagons are equal when they have the same coordinates.

Main features:

- **Scalar** and **vector** operations
 - $Hex + Hex$
 - $Hex - Hex$
 - $Hex * N$
 - Hex / N
- **Distance** between two hexagons
- **Lerp** between two hexagons
- **IsValid** validates hexagon coordinates
- **Rounding** to a hexagon
 - `GetNearestHexagon`
 - `GetNearestHexagonRound`
 - `GetNearestHexagonFloor`

Geometry 2D

`Rodak.Hexagons.HexGeometry`

The geometry extension adds functions used for calculating 2D coordinates around the hexagon.

Main features:

- `GetCenter` gets the center 2D coordinate
- `GetCorner` gets a corner 2D coordinate
- `GetSide` gets a side 2D coordinate
- `RotateClockwise` rotates the hexagon around the origin clockwise
- `DirectionsClockwise` are clockwise ordered directions/neighbors around the origin hexagon
- `GetAbsoluteIndex` loops the corner/side hexagons index
- `GetHexagonAt` gets the hexagon that contains the 2D coordinate

Geometry 3D

`Rodak.Hexagons.HexGeometry3D`

The geometry 3D extension adds functions used for translating the 2D coordinates to and from a 3D plane.

Uses a default `PlacementPlane` when none is specified.

Main features:

- `GetCenter3D` gets the center 3D coordinate
- `GetCorner3D` gets a corner 3D coordinate
- `GetSide3D` gets a side 3D coordinate
- `GetHexagonAt` gets the hexagon that contains the 3D coordinate

Navigation

`Rodak.Hexagons.HexNavigation`

These scripts are useful in navigation.

Relations The relations extension adds functions used for finding related hexagons.

Main features:

- `GetNeighbors` gets all of the neighbors of the hexagon
- `GetDiagonals` gets all of the diagonals of the hexagon
- `GetRange` calculates all hexagons in the movement range of n
- `GetRing` calculates all hexagons in a ring of a radius
- `GetSpiral` calculates all hexagons in a spiral up to a radius

Line The `HexagonLine` is a straight line segment between a start and an end hexagon.

Main features:

- `Length` is the line length
- `Indexer [index]` of a hexagon on this line
- `Start` is the start hexagon
- `End` is the end hexagon

Grid

`Rodak.Hexagons.HexGrid`

The `HexagonGrid<TTile>` is a generic definition of a 2D grid with a fixed size.

These scripts define:

- `hexagonPosition` as a position of a tile

Main features:

- `Indexer` with `[hexagonPosition]` as the index
- `TryGetValue` tries to get the value at the `hexagonPosition`
- `TrySetValue` tries to set the value at the `hexagonPosition`
- `ContainsHexagon` checks if a hexagon is contained in this grid
- `ForEach` executes a `System.Action` for every tile
- `Hexagons` is a list of all contained `hexagonPositions`

Map

`Rodak.Hexagons.HexMap`

The `AHexagonMap<TTile, GChunk>` is a generic abstract definition of an infinite chunk-based map. It requires a tile and chunk type.

The `HexagonMap<TTile>` is a generic definition of a basic map that uses the `HexagonChunk<TTile>`, which is a generic definition of a basic chunk.

These basic definitions are useful for simpler use cases. It is recommended to implement your own map and chunk definition; they should inherit from `AHexagonMap<TTile, GChunk>` and `HexagonChunk<TTile>` respectively.

These scripts define:

- `hexagonPosition` as a position of a tile
- `chunkPosition` as a position of a chunk

Main features of the base map:

- Indexer of chunks with `[chunkPosition]` as the index
- `GetOrCreateChunk` tries to get the chunk at the `chunkPosition` and creates a new one if it fails
- `TryGetChunk` tries to get the chunk at the `chunkPosition`
- `TryGetHexagonsChunk` tries to get the chunk at the `hexagonPosition`
- `TryRemoveChunk` tries to remove the chunk at the `chunkPosition`
- `GetChunkPosition` calculates the `chunkPosition` from a contained `hexagonPosition`
- `ContainsChunkPosition` checks if a chunk position is contained in this map
- `ForEach` executes a `System.Action` for every chunk

Main features of the base chunk:

- Indexer of tiles with `[hexagonPosition]` as the index
- `TryGetValue` tries to get the tile at the `hexagonPosition`
- `SetValue` tries to set the tile at the `hexagonPosition`
- `ContainsHexagon` checks if a tile position is contained in this map
- `ForEach` executes a `System.Action` for every tile

Editor

`Rodak.Hexagons.HexEditor`

Since the core definition of a `Hexagon` is immutable, a different class is needed in the inspector. This is where `EditableHexagon` comes in.

`EditableHexagon` should not be used outside of the inspector.

It defines implicit conversions to `Hexagon` and back, so that it can be easily used. It has a built-in auto correct that ensures $Q + R + S = 0$. Also, it supports custom attributes to further improve its use.

The attribute `[HexAutoCorrect(HexAutoCorrectMode)]` overrides the default auto correct behavior.

- `HexAutoCorrectMode.TwoComponent` - Both `Q` and `R` correct `S`. `S` corrects `Q`. (default)
- `HexAutoCorrectMode.Staggered` - Each component corrects the next one. $Q \Rightarrow R \Rightarrow S \Rightarrow Q$

The attribute `[HexReadOnly]` makes the `EditableHexagon` appear read-only in the inspector.

Debug

`Rodak.Hexagons.HexDebug`

These scripts add functions that help to quickly debug in the scene. They use `UnityEngine.Debug.DrawLine` to render hexagons in the scene.

Main features:

- `DebugDraw` draws the hexagon outline and optionally the individual triangles

IO

`Rodak.Hexagons.HexIO`

The IO extension adds functions used for serializing and deserializing hexagons and hexagonal data structures.

Binary Adds simple `System.IO.BinaryWriter` and `System.IO.BinaryReader` extension methods for writing/reading hexagons from binary.

Main features:

- `Write` writes a hexagon to binary
- `ReadHexagon` reads a hexagon from binary

Utils

`Rodak.Hexagons.HexIO`

Placement Plane Definition of a plane in 3D space.

Main features:

- `LayOnPlane` translates a 2D coordinate to a 3D coordinate on that plane
- `Get2DPosition` translates a 3D coordinate on that plane to a 2D coordinate
- Predefined axis planes
 - `XYPlane`
 - `XZPlane`
 - `YZPlane`

Mesh Builder Manages vertices, UVs, and submesh triangles.

Easily convertible to the `UnityEngine.Mesh` with `ToMesh`.